

```
impl Keywords {
  pub fn new() -> Self {
    // The following list contain keywords that are named meme_style
    // Last word will be in മലയാളലിപി(Malayalalipi)
    let list = keywordize!(
      ["pwoli_sadhanam", "pwoli_sanam", "pwoli_saanam", "poli_sadhanam",
       "poli_sanam", "poli_saanam", "പൊളിസാധനം", ] => TokenType::Declaration,
      ["address_thada", "അഡ്രസ് താടാ"] => TokenType::InputString,
      ["number_thada", "നമ്പർ താടാ"] => TokenType::InputNumber,
      ["dhe_pidicho", "ദേ പിടിചോ"] => TokenType::Write,
      ["seriyano_mwone", "seriyano", "ശെരിയാണോ മോനെ", "ശെരിയാണോ"] => TokenType::If,
      ["seri_allel", "ശെരി അല്ലേൽ"] => TokenType::Else,
      ["repeat_adi", "റിപീറ്റടി"] => TokenType::Loop,
      ["veluthane", "veluthanekil", "veluthanekil", "valuthanekil",
       "valuthanekil", "valuthane", "വലുതാണെ", "വലുതാണെങ്കിൽ", "വലുതാണെങ്കിൽ", ] => TokenType::GreaterThan,
      ["same_alle", "same_alle", "സെയിറ്റ അല്ല"] => TokenType::NotEqual,
      ["cheruthane", "cheruthanekil", "cheruthanekil", "charuthane",
       "charuthanekil", "charuthanekil", "ചെറുതാണെങ്കിൽ", "ചെറുതാണെങ്കിൽ", ] => TokenType::LessThan,
      ["same_aane", "സെയിറ്റ ആണെ"] => TokenType::EqualTo,
      ["um", "ഉം"] => TokenType::Um,
      ["ne_kal", "നേകാൾ"] => TokenType::Nekal,
      ["koduthek", "കൊടുത്തേക്", "കൊടുത്തേക്"] => TokenType::Return
    );
  }
  Self { list }
} fn new
} impl Keywords
```

Figure 2: Keyword mapping in Malluscript

```
Statement:Statement = {
  <a:@L> Declaration <id:Expression> ";" <b:@R> => Statement::Declaration((a,b),id),
  <a:@L> <l:Expression> "=" <r:Expression> ";" <b:@R> => Statement::Assignment((a,b),l,r),
  <a:@L> Write <e:Expression> ";" <b:@R> => Statement::Write((a,b),e),
  <a:@L> If <condition:Conditional> "{" <s:SourceUnit> "}" <b:@R> => Statement::Conditional((a,b),condition,s,None),
  <a:@L> If <condition:Conditional> "{" <s:SourceUnit> "}" Else "{" <f:SourceUnit> "}" <b:@R> => Statement::Conditional((a,b),condition,s,Some(f)),
  <a:@L> <name:Expression> "(" <params:Expressions> ")" "{" <s:SourceUnit> "}" <b:@R> => Statement::FunctionDeclaration((a,b),name,params,s),
  <a:@L> Return <expr:Expression> ";" <b:@R> => Statement::Return((a,b),expr),
  <a:@L> <expr:Expression> ";" <b:@R> => Statement::EmptyExpression((a,b),expr),
  <a:@L> Loop <condition:Conditional> "{" <s:SourceUnit> "}" <b:@R> => Statement::Loop((a,b),condition,s)
};
```

Figure 3: Parser grammar in Malluscript

Writing a parser

The parser's main job is to take the input provided by the lexer (which is a stream of tokens) and arrange it into a tree-like structure called a parse tree. The parse tree represents the hierarchical structure of the input and shows how the different tokens fit together to form a complete program.

The parser uses a set of rules known as grammar. Using this grammar, parser determines how to arrange the tokens. Malluscript's parser is written using a parser generator called LALRPOP.

LALRPOP, as quoted by its author, is a Rust parser generator framework with usability as its primary goal. LALRPOP lets us write clean and readable context-free grammars, and we define how Malluscript's code should be interpreted and how it should look. For example, consider the statement parser of Malluscript in LALRPOP shown in Figure 3. It closely resembles the context-free grammar we usually write for theoretical compiler design problems. On compilation (compilation of the interpreter), LALRPOP converts these CFG rules to Rust code. And

using our parser, we parse our tokens generated in the previous step, and generate an abstract syntax tree (AST).

Writing an executor

Now our lexer-converted tokens are built into an AST, which represents how the code should be executed. The job of our executor is to simply navigate through AST nodes and do the necessary operations mentioned by them. For example, Figure 4 shows executor code for a declaration statement in Malluscript.

The code in Figure 4 is a declaration statement. The executor takes a symbol and checks if that symbol exists on the symbol table. If it does, we throw an error 'Symbol already defined'; otherwise, we add the symbol to the symbol table. A simple program to find factorial in Malluscript will look like what's shown below:

```
pwoli_sadhanam num;
pwoli_sadhanam factorial;
dhe_pidicho "Input number:";
```

Continued to page...87